

Project: Serverless Image/Video Processing

Services: S3, Lambda, Step Functions, Rekognition, SNS

Goal: Automatically process images or videos uploaded to S3 (e.g., resize, label detection).

Skills: Event-driven architecture, serverless orchestration.

Definition: Serverless Image/Video Processing is a cloud-based project that automatically processes images or videos uploaded to an S3 bucket. Using AWS Lambda for compute, Rekognition for media analysis, Step Functions for orchestrating multi-step workflows, and SNS for notifications, the system detects labels, resizes or transcodes files, and sends alerts when processing is complete — all without managing any servers.

Scope of the Project:

- Automatically triggers media processing when images or videos are uploaded to Amazon S3.
- Uses a fully serverless, event-driven architecture without managing any servers.
- Orchestrates multiple processing steps using AWS Step Functions.
- Performs media analysis such as label detection and content recognition using Amazon Rekognition.
- Stores processed media files securely in an output S3 bucket.
- Sends real-time notifications about processing status through Amazon SNS.
- Supports scalability and high availability using AWS-managed services.
- Designed to be cost-effective and compatible with AWS Free Tier for learning and academic use.

Methodology:

1.Event-Driven Serverless Orchestration Method (Manual Setup)

- Uses S3, Lambda, Step Functions, Rekognition, and SNS.
- Provides hands-on understanding of event-driven serverless media processing and workflow orchestration.
- Enables step-by-step configuration and testing of file upload, media analysis, processing logic, and notifications using coordinated Lambda functions.
- Utilizes Step Functions to manage sequential execution, retries, and error handling across multiple processing stages.
- Allows quick deployment within AWS Free Tier for demos or academic projects with controlled workloads.
- Cost-effective and ideal for small-scale, proof-of-concept, or learning-focused projects.

2.Template-Based Serverless Orchestration Method (Automated Setup)

- Uses CloudFormation templates to define S3 buckets, Lambda functions, Step Functions workflows, Rekognition permissions, and SNS topics.
- Ensures consistent and repeatable deployment of all serverless components.
- Reduces manual configuration errors and setup time.
- Suitable for production-level or multi-environment projects.

AWS Services used in the project:

1. Amazon S3

- **Definition:** Amazon S3 is an object storage service that provides secure, scalable, and durable storage for files.
- **Role in the Project:** Acts as the entry point for the system where users upload images or videos and stores processed output files.
- **Purpose:** To store input media files and processed results reliably and trigger processing workflows automatically.

2.AWS Lambda

- **Definition:** Lambda is a serverless compute service that runs code in response to events without managing
- **Role in the Project:** Executes media processing logic such as validation, resizing, and calling Rekognition APIs.
- **Purpose:** To perform processing tasks automatically when triggered by S3 or Step Functions.

3. AWS Step Functions

- **Definition:** AWS Step Functions is a serverless workflow service used to coordinate multiple AWS services into a sequence of steps.
- **Role in the Project:** Manages and orchestrates multiple Lambda functions in a defined processing workflow.
- **Purpose:** To ensure reliable execution, sequencing, retries, and error handling of media processing steps.

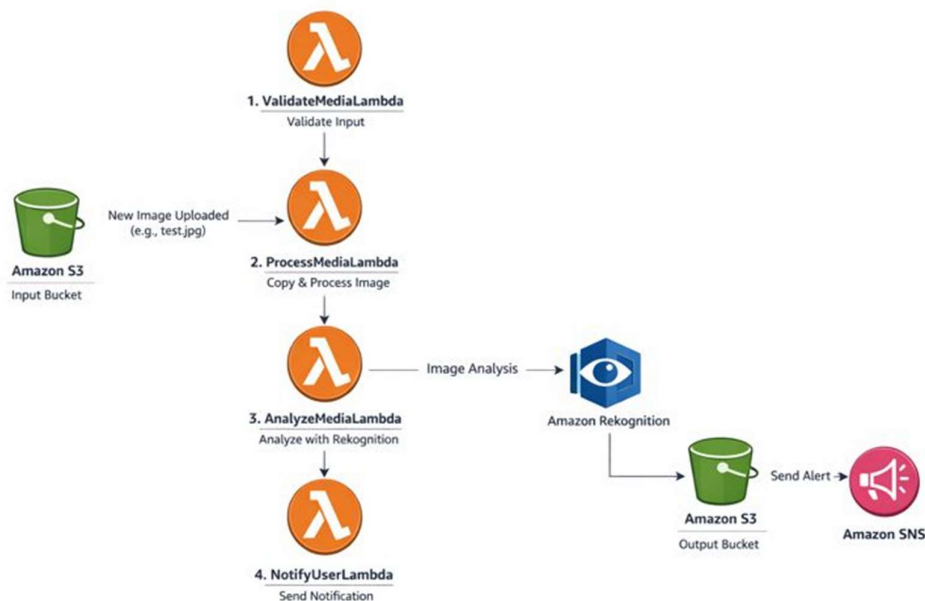
4. Amazon Rekognition

- **Definition:** Amazon Rekognition is a machine learning service that analyzes images and videos for objects, labels, faces, and scenes.
- **Role in the Project:** Analyzes uploaded media to detect labels or content automatically.
- **Purpose:** To extract meaningful insights from images or videos without building custom ML models.

5. Amazon SNS (Simple Notification Service)

- **Definition:** Amazon SNS is a messaging service that sends notifications to subscribers through email, SMS, or other endpoints.
- **Role in the Project:** Sends notifications when media processing is completed or if errors occur.
- **Purpose:** To inform users or administrators about the processing status in real time.

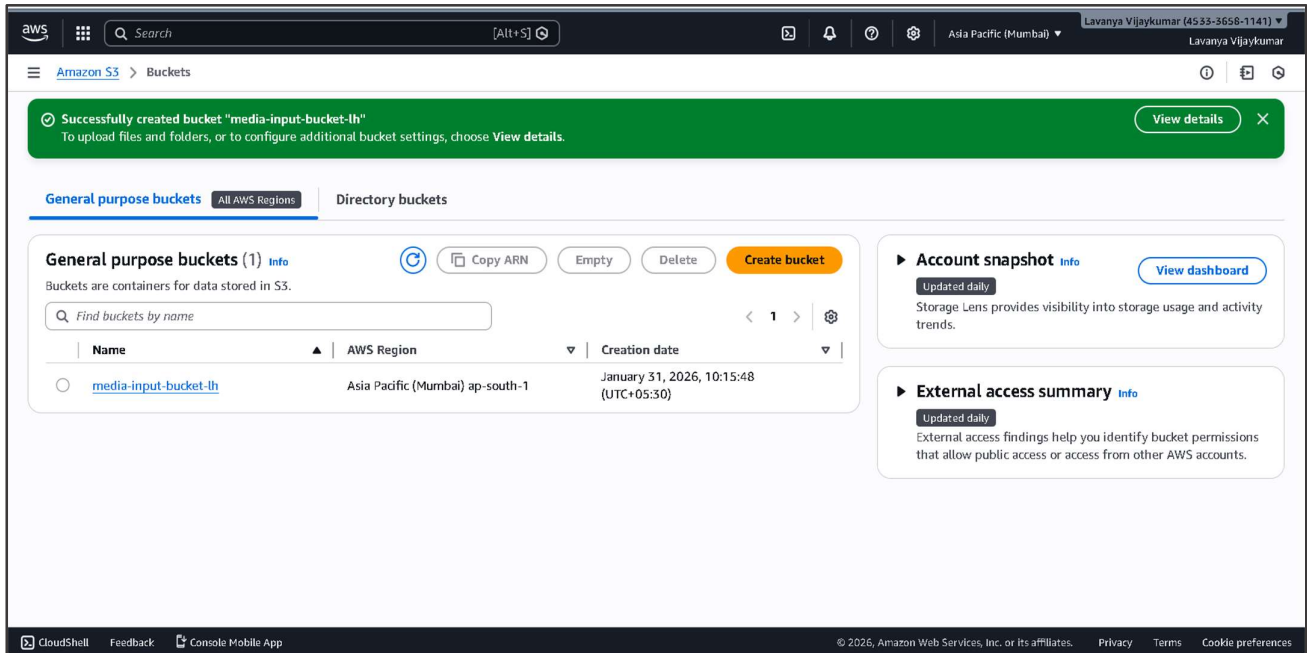
Architecture Diagram:



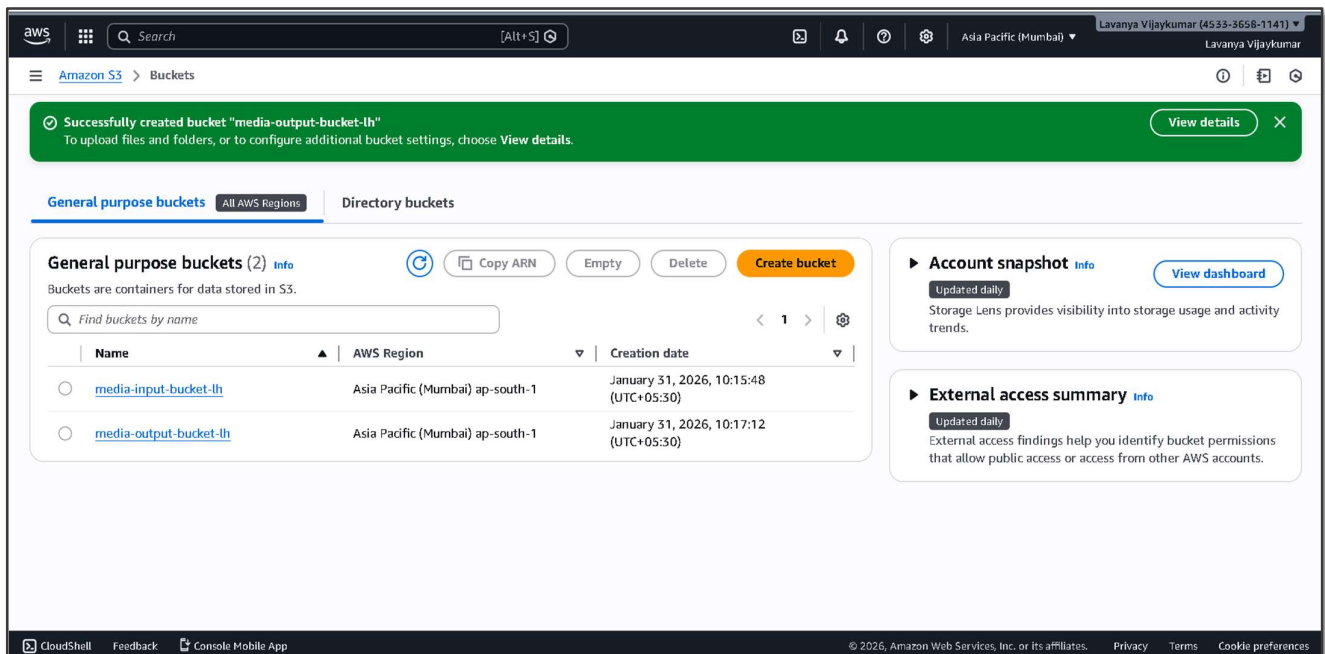
Project Implementation Steps;

STEP 1: Create S3 Buckets

- Go to S3 → Create bucket
- Name: media-input-bucket-lh
- Block public access: ON
- Create bucket

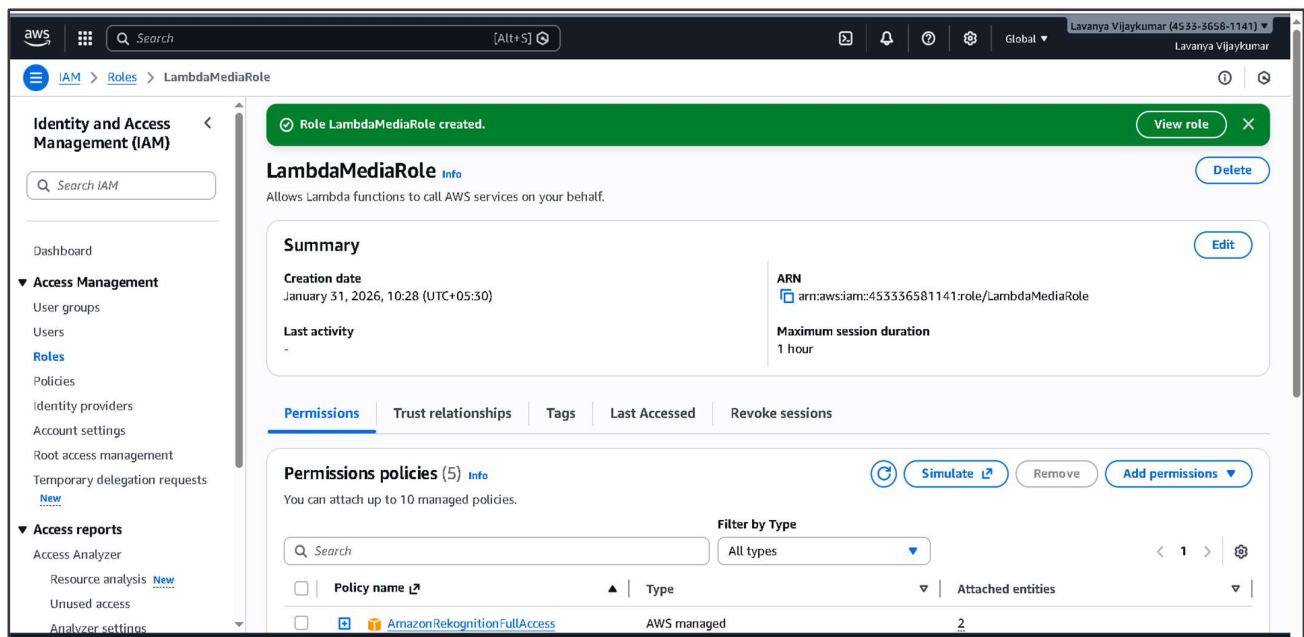


- Go to S3 → Create bucket
- Name: media-input-bucket-lh
- Block public access: ON
- Create bucket



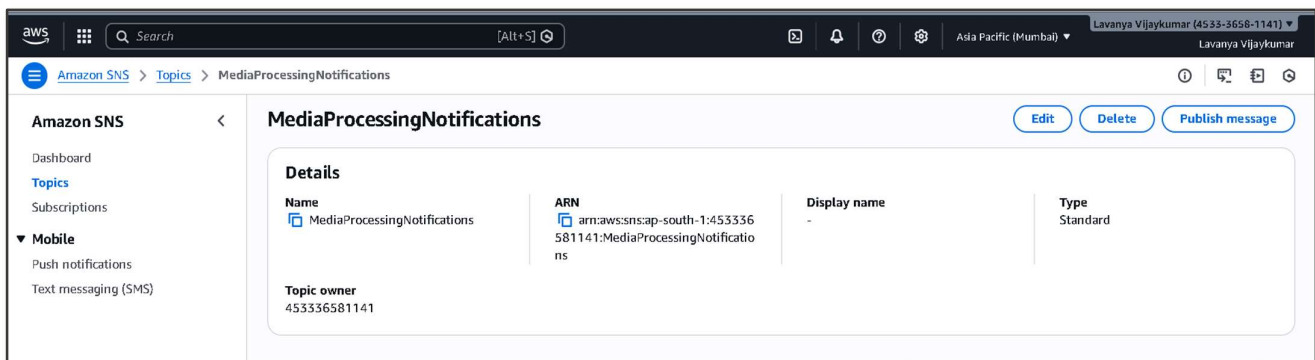
STEP 2: Create IAM Role for Lambda

- Go to IAM → Roles → Create Role
- Trusted entity: AWS service → Lambda
- Attach permissions
 1. AWSLambdaBasicExecutionRole
 2. AmazonS3FullAccess
 3. AmazonRekognitionFullAccess
 4. AWSStepFunctionsFullAccess
 5. AmazonSNSFullAccess
- Role name=LambdaMediaRole
- Create role

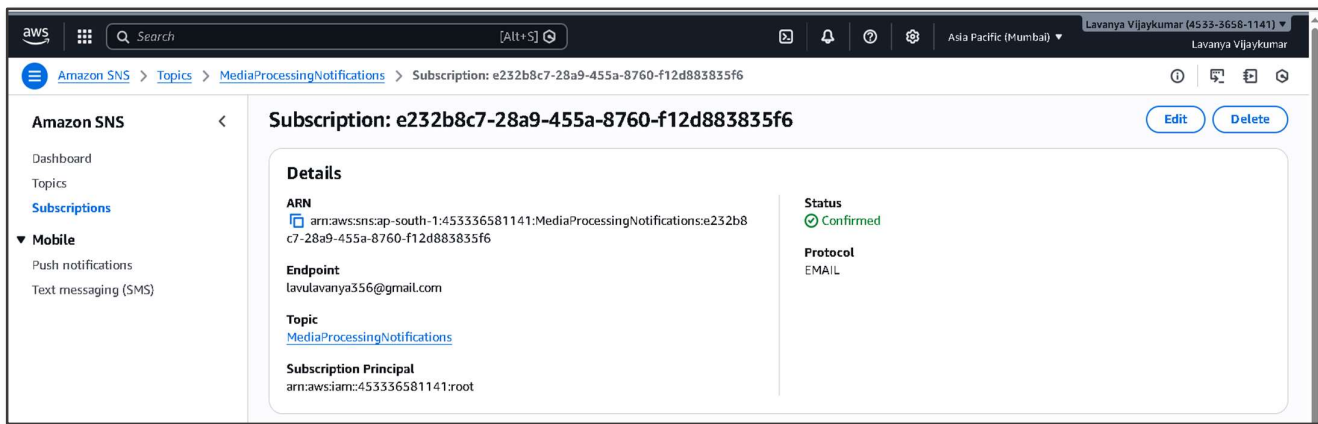


STEP 3: Create SNS Topic

- Go to SNS
- Create topic
- Topic name= MediaProcessingNotifications
- Create Topic



- Create Subscription

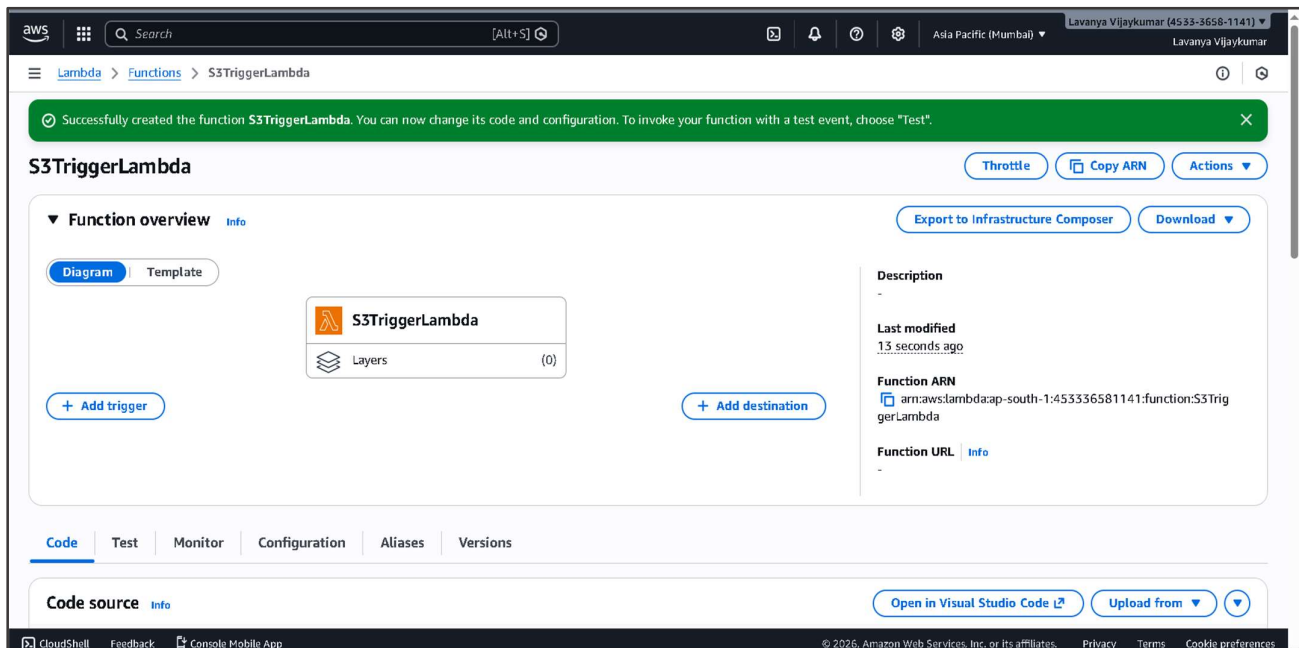


STEP 4: Create Lambda Functions

- Need 5 Lambda functions:

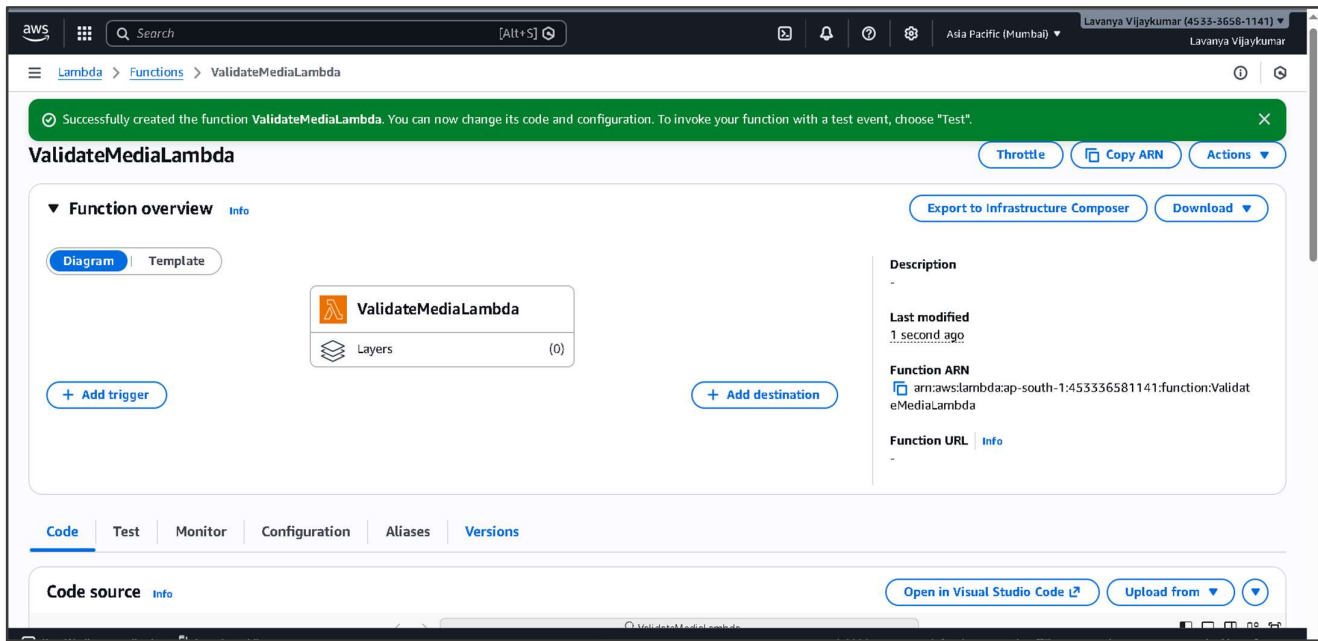
1.Lambda Name: S3TriggerLambda

- Purpose: Starts Step Functions automatically when file uploaded to S3
- Runtime: Python 3.10
- Role: LambdaMediaRole
- Add code

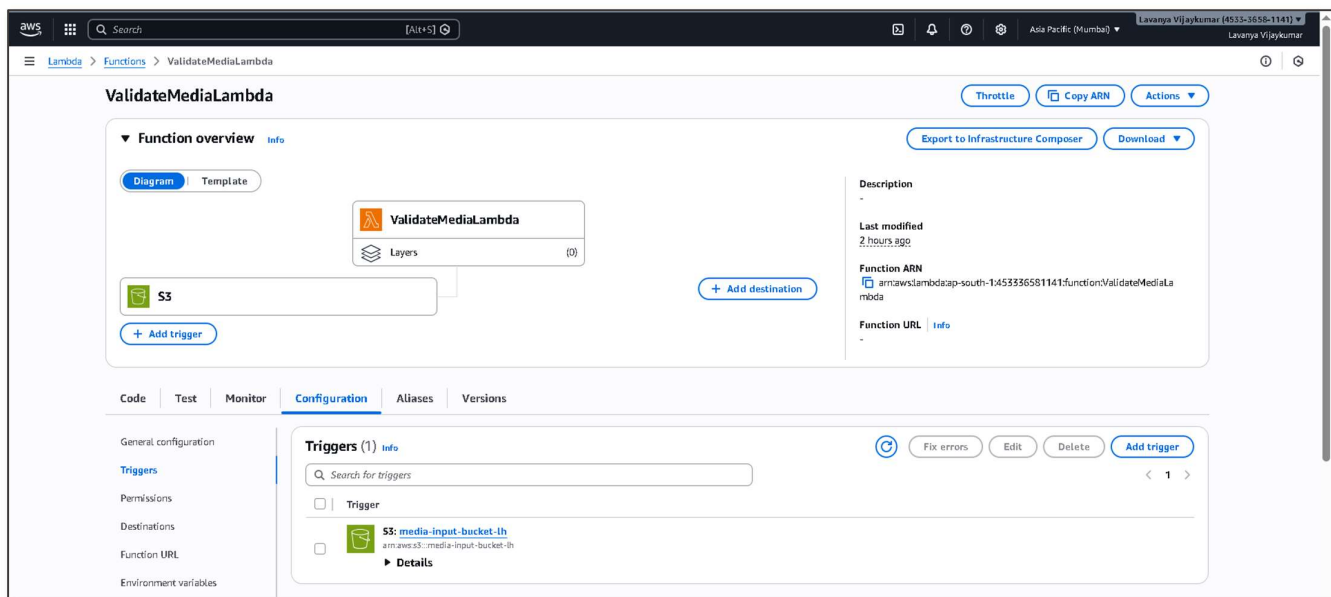


2.Lambda Name:ValidateMediaLambda

- Purpose: Ensure uploaded file is valid (jpg, png, mp4, etc.)
- Runtime: Python 3.10
- Role: LambdaMediaRole

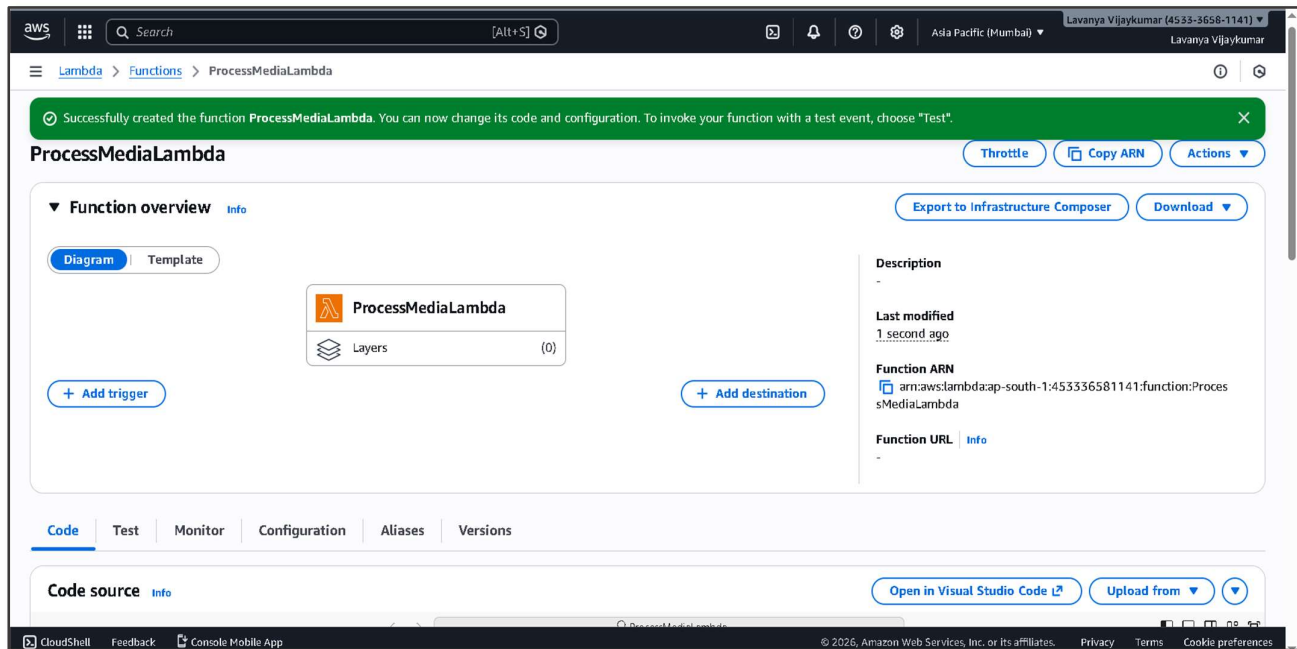


- Add trigger = S3 Bucket



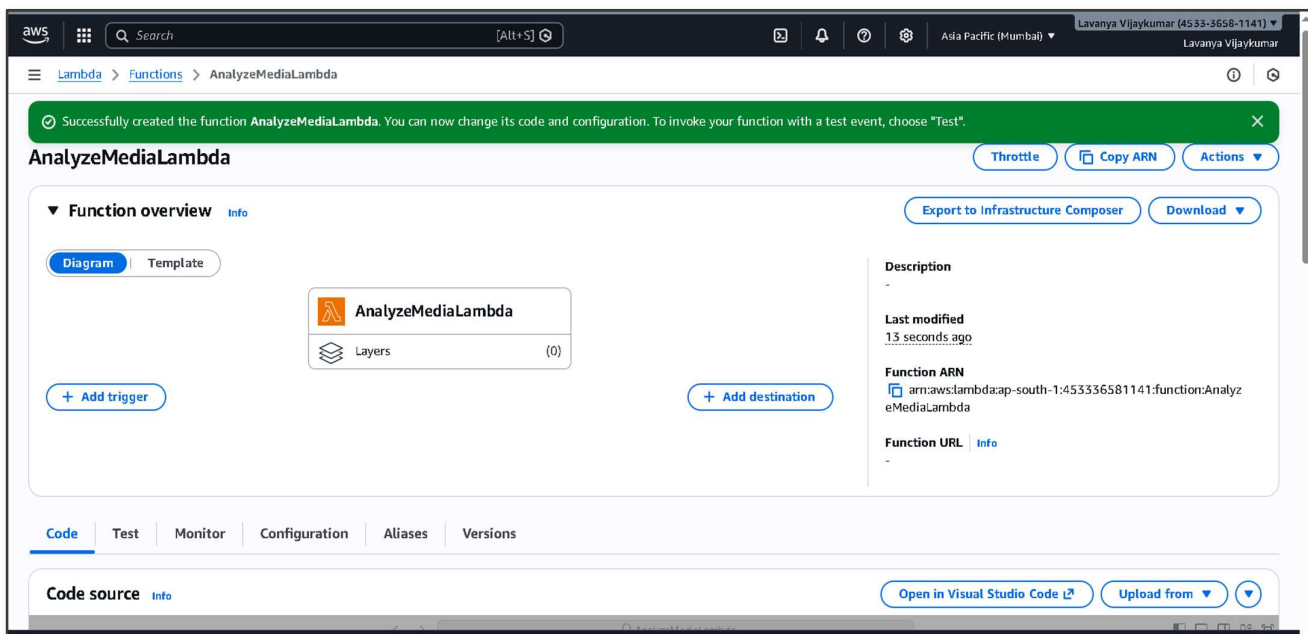
3.Lambda Name:ProcessMediaLambda

- Purpose: Resize/convert media & store in output bucket
- Runtime: Python 3.10
- Role: LambdaMediaRole



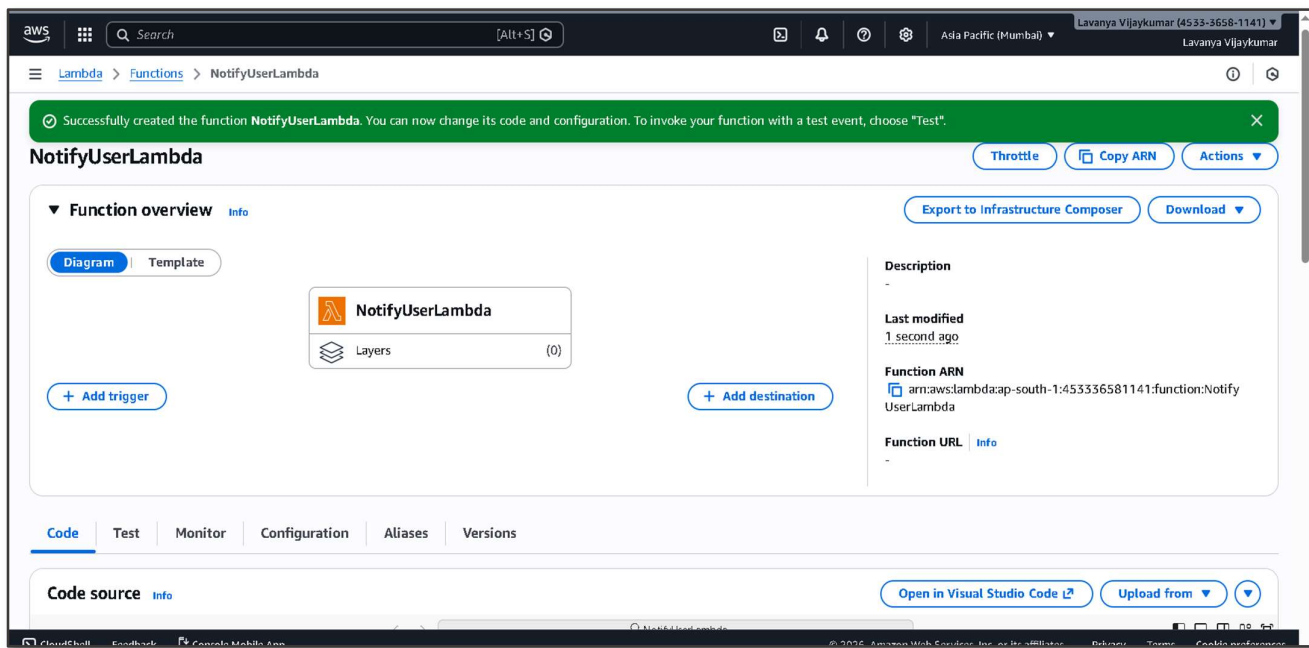
4.Lambda Name:AnalyzeMediaLambda

- Purpose: Detect labels using Rekognition
- Runtime: Python 3.10
- Role: LambdaMediaRole



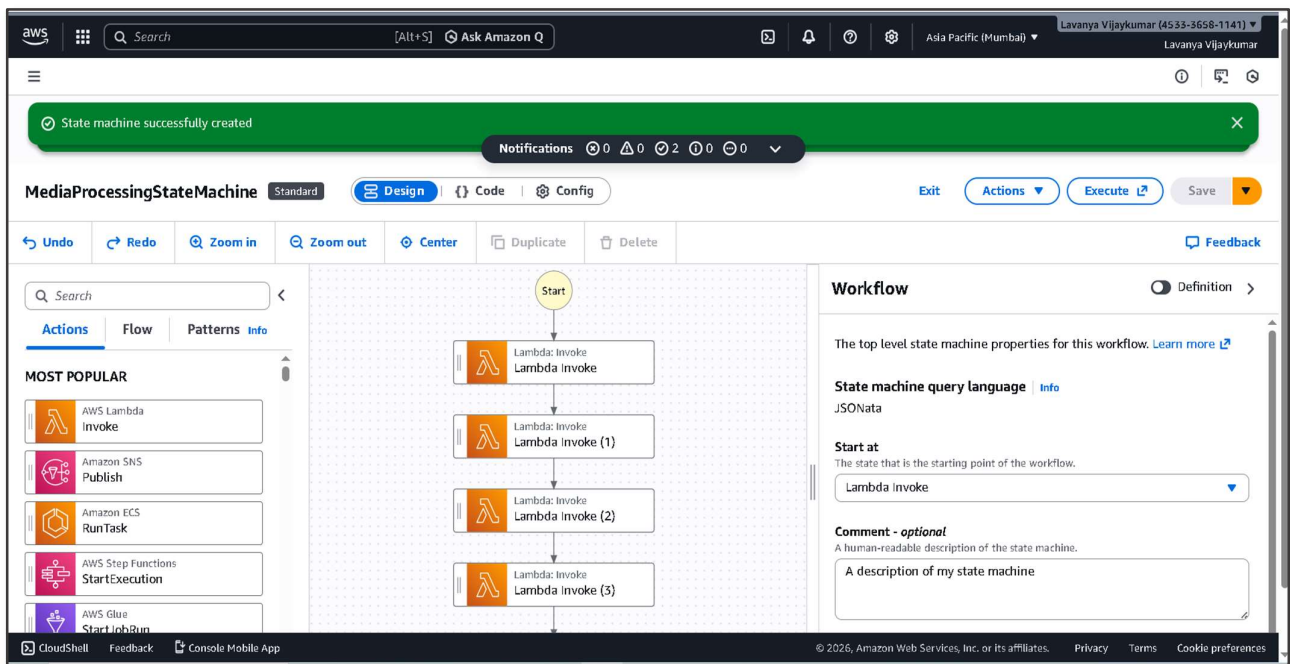
5.Lambda Name:NotifyUserLambda

- Purpose: Send SNS/email notification after processing
- Runtime: Python 3.10
- Role: LambdaMediaRole

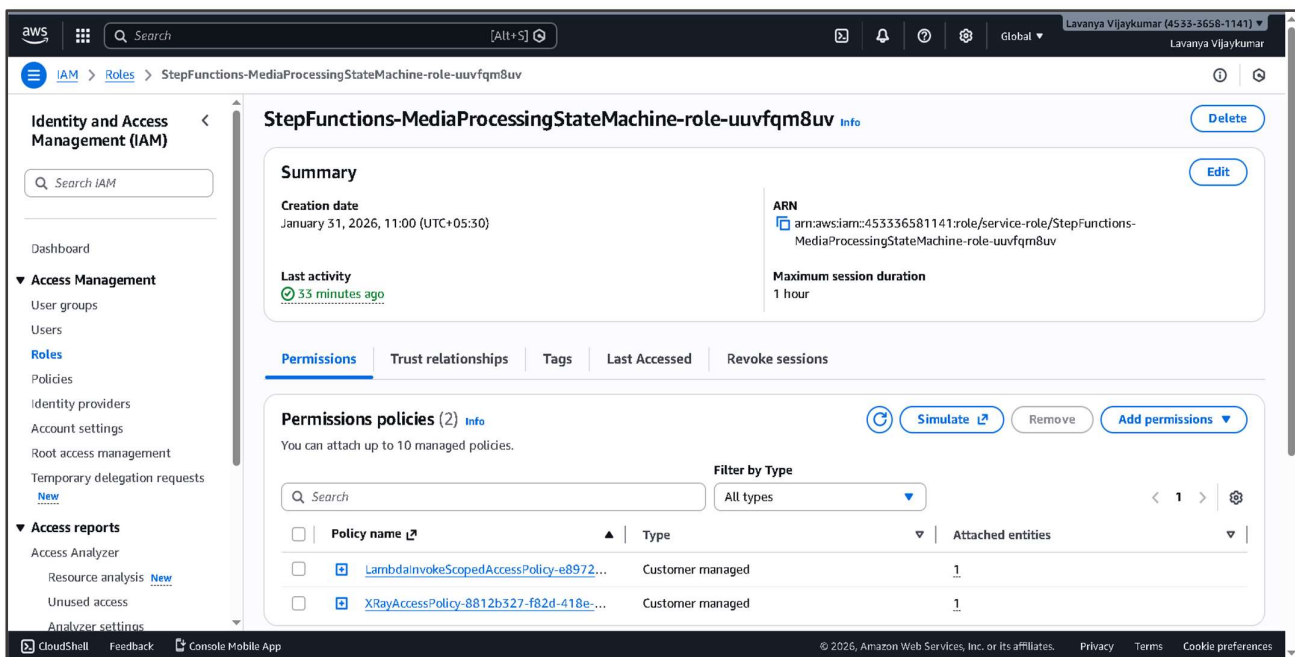


STEP 4: Create Step Functions State Machine

- Go to Step Functions
- Click Create state machine
- State machine= MediaProcessingStateMachine
- Type=Standard
- Click next
- Now inside the Workflow Studio (drag & drop screen).
- **Add First State**
 1. Drag and drop lambda invoke inside workflow.
 2. Select Lambda function: Select **ValidateMediaLambda**
- **Add Second State**
 1. Drag and drop lambda invoke after lambda invoke 1 inside workflow
 2. Select Lambda function: **ProcessMediaLambda**
- **Add Third State**
 1. Drag and drop lambda invoke after lambda invoke 2 inside workflow
 2. Select Lambda function: **AnalyzeMediaLambda**
- **Add Final State**
 1. Drag and drop lambda invoke after lambda invoke 3 inside workflow
 2. Select Lambda function: **NotifyUserLambda**
- Click on Deploy
- Confirm on automatically creating role by step functions.

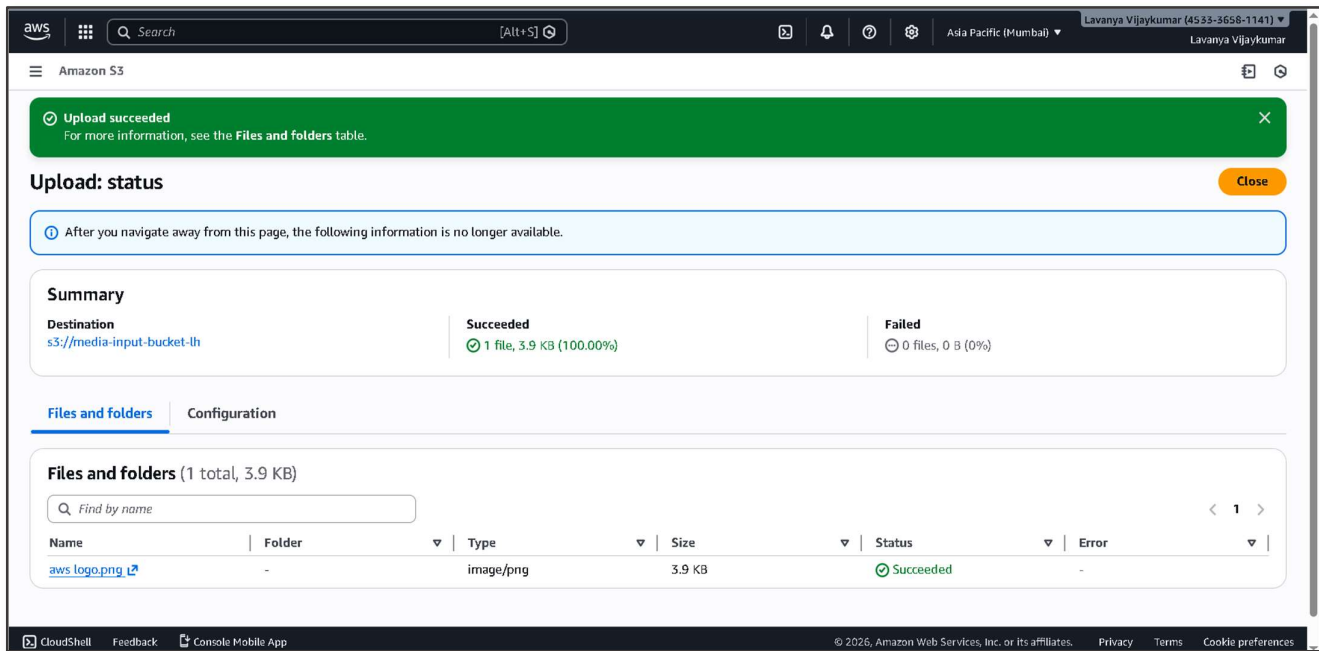


- Role created by step functions.



STEP 5: Add object in s3 input Bucket

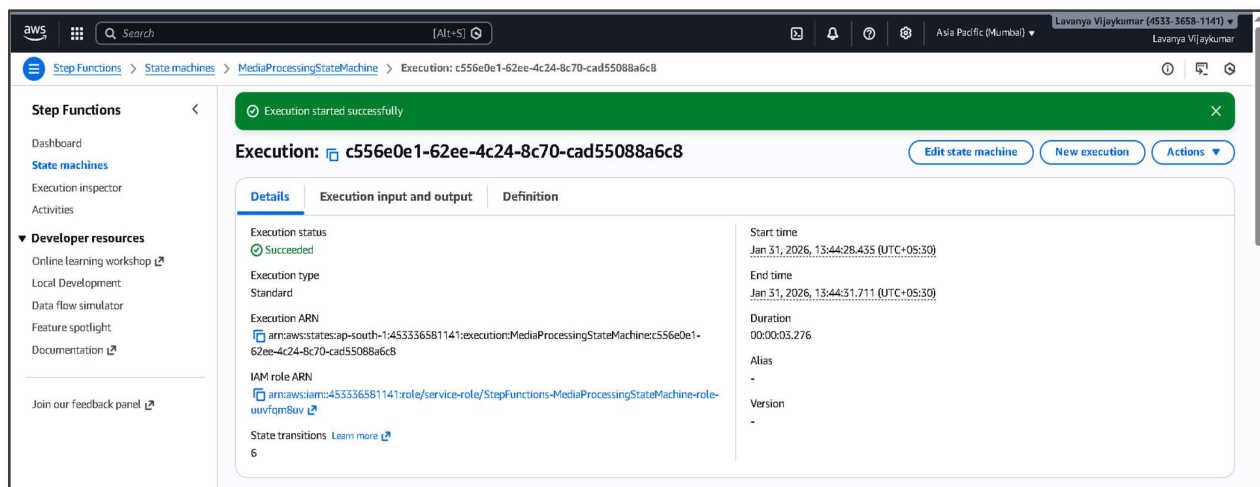
- Go to S3
- Click on media-input-bucket-lh
- Click on upload objects
- Add upload image object=aws logo.png
- Click on upload



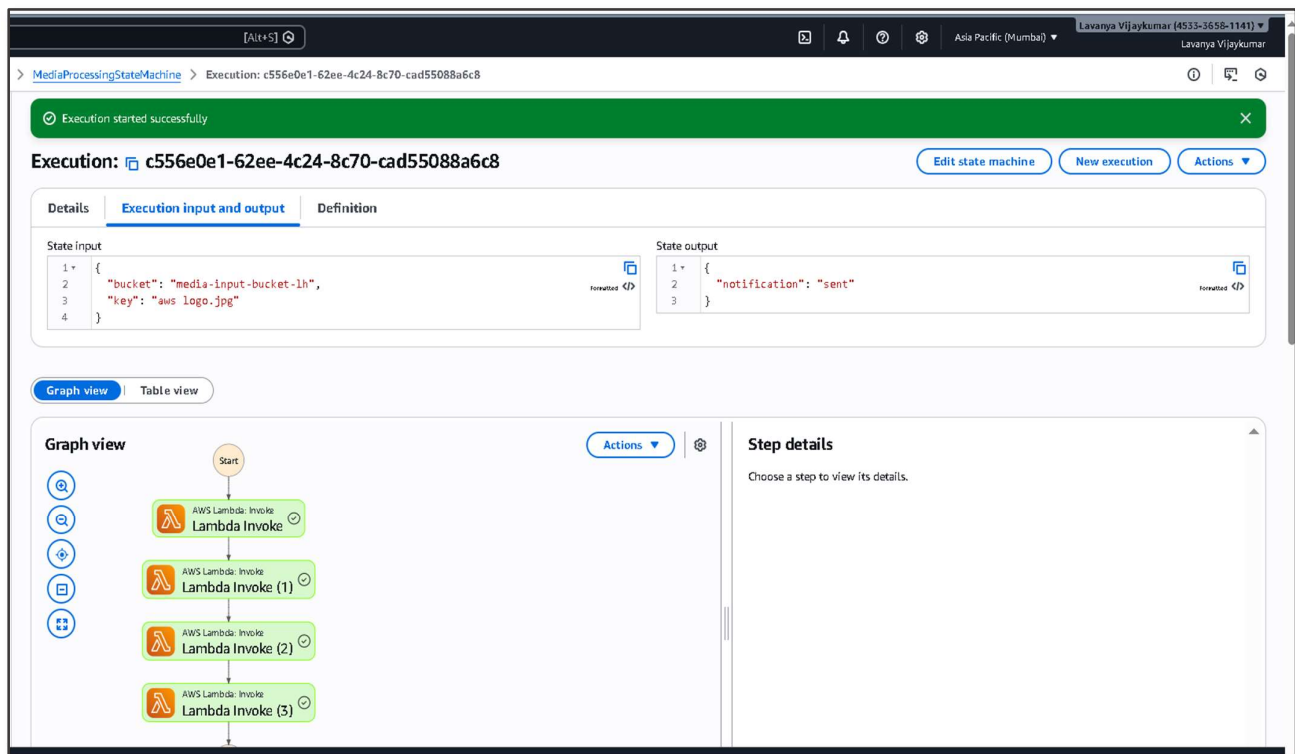
STEP 6: In Step Functions Start execution

- Go to step function
- Click on start execution
- In input column add


```
{
  "bucket": "media-input-bucket-lh",
  "key": "aws.logo.jpg"
}
```
- Start execution
- Will get Succeeded in execution status

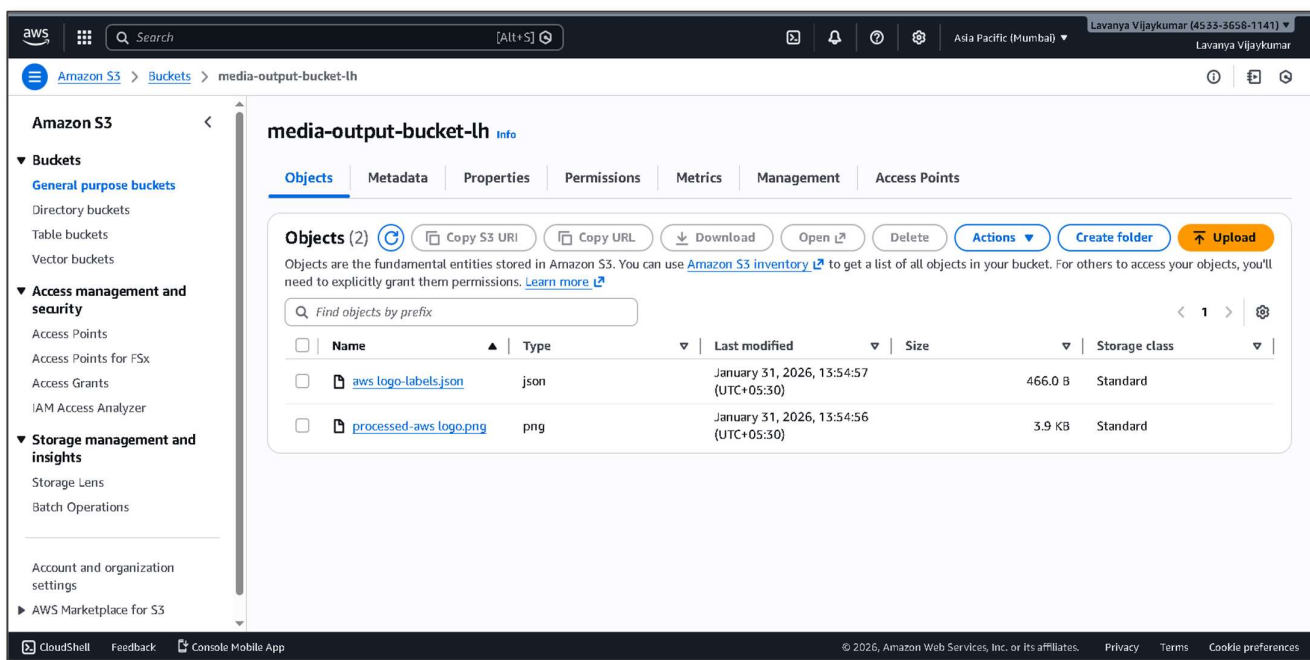


- Execution input and output

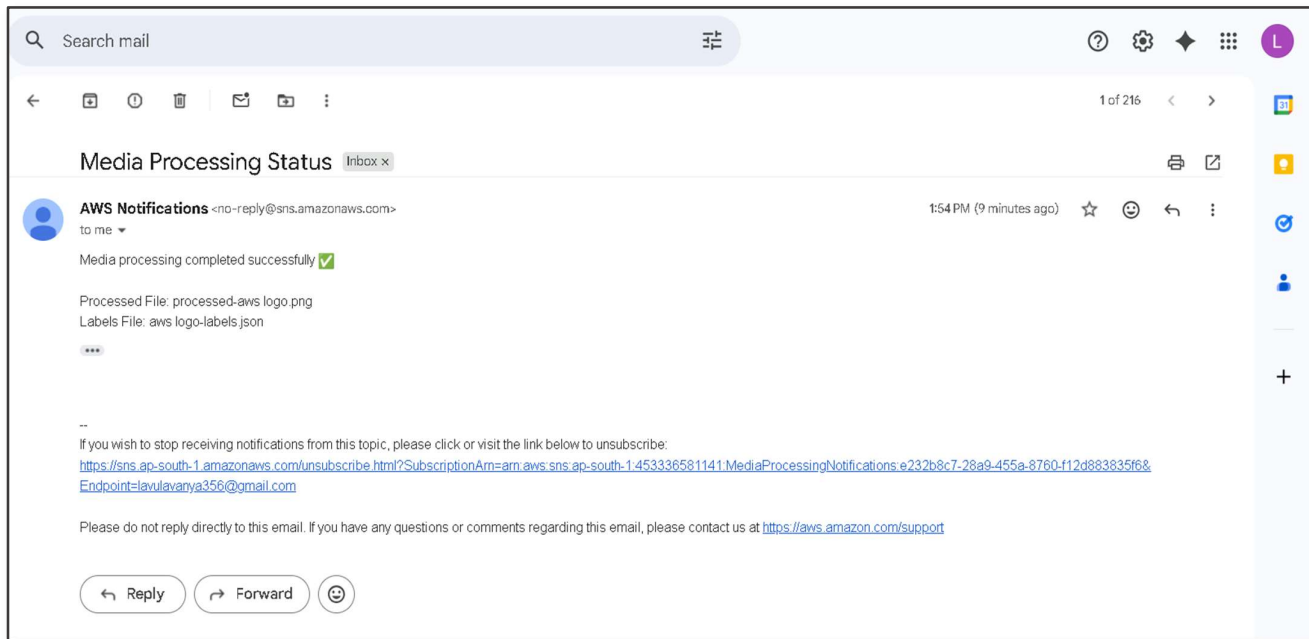


STEP 7: Check media-output-bucket-lh for output

- Go to s3
- Click on media-output-bucket-lh
- Processed image and label will be created inside bucket



- Check email: mail will be received contains like success message and both file names.



Troubleshooting:

Issue 1: Lambda execution failed with error:

"Key Error": "Records"

Cause: Lambda function was written to handle S3 event triggers, but was invoked by AWS Step Functions, which passes a different JSON structure.

Solution:

- Modified Lambda code to handle Step Functions input format
- Added conditional checks for bucket and key instead of Records

Issue 2: Email notification showed:

Processing completed: { "status": "ERROR", "message": "Missing bucket or key" }

Cause:

- Step Functions was not passing output from one state to the next correctly
- Incorrect ResultPath or OutputPath configuration

Solution:

- Set InputPath, ResultPath, and OutputPath to \$ for all states
- Ensured consistent output keys between Lambda functions (bucket, processed_key, labels_file)

Conclusion

This project successfully implements an automated serverless media processing system using AWS cloud services such as Amazon S3, AWS Lambda, AWS Step Functions, Amazon Rekognition, and Amazon SNS. The system processes images uploaded to cloud storage without the need for server management, ensuring scalability and efficiency.

The application follows an event-driven architecture, where uploaded media triggers a sequence of Lambda functions orchestrated using AWS Step Functions. The workflow validates and processes the image, performs label detection using Amazon Rekognition, stores the processed files and metadata in Amazon S3, and finally sends notifications to the user.

By integrating Amazon Rekognition, the system adds intelligent image analysis capabilities, enabling automatic identification of objects and scenes within images. The use of serverless services reduces operational overhead, supports on-demand scaling, and makes the solution cost-effective and suitable for academic projects, prototypes, and small-scale applications.